

Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
Кафедра «Вычислительная техника»

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №2

Умножение матриц на CUDA

ПРОЕКТ: экспериментальная часть ожидает запуска

Преподаватель

подпись, дата

С.А. Тарасов

Студент

КИ23-06Б 032320481

подпись, дата

А.А. Платонов

Студент

КИ23-06Б 032321113

подпись, дата

Н.С. Бученик

Красноярск 2026

ВВЕДЕНИЕ

Цель работы - закрепить базовые навыки программирования CUDA и освоить работу с двумерными сетками нитей. Требуется реализовать умножение матриц с архитектурой Data + View, проверить результат по Eigen и исследовать время вычислений.

Разработаны шаблонные классы Data, MatrixView и Matrix. Для умножения используется наивный CUDA-кernels без разделяемой памяти. Каждая нить вычисляет один элемент результата. Исходные данные располагаются в памяти построчно (row-major).

Проверка включает все 343 сочетания m, n, k из множества $\{1, 2, 3, 127, 128, 129, 512\}$. Производительность исследуется для квадратных матриц порядков 16, 32, 64, 128, 256, 512, 1024.

Полная реализация, тесты, скрипты запуска и построения отчета:

<https://github.com/NikitaBuchenik/hsys/tree/main/second>

Статус. Этот документ описывает подготовленную реализацию и методику. CUDA-сборка, тесты на GPU и измерения еще не выполнены. Числовые результаты и экспериментальные графики намеренно отсутствуют.

1 Разработка программного решения

1.1 Владение памятью: Data

Data непосредственно владеет массивом в глобальной памяти GPU. Конструктор выделяет память через `cudaMalloc`, деструктор освобождает ее через `cudaFree`. Копирование Data выполняет глубокую копию, перемещение передает указатель. Оператор копирующего присваивания использует временный объект и `swap`.

```
explicit Data(std::size_t size) : size_(size) {
    if (size > std::numeric_limits<std::size_t>::max() / sizeof(AtomT))
        throw std::length_error("Data byte count overflow");
    if (size_) CUDA_CHECK(cudaMalloc(reinterpret_cast<void*>(&data_), size_ *
sizeof(AtomT)));
}
Data(const Data& other) : Data(other.size_) {
    // Release the allocation explicitly if the deep copy fails.
    if (size_) {
        const auto error = cudaMemcpy(data_, other.data_, size_ * sizeof(AtomT),
cudaMemcpyDeviceToDevice);
        if (error != cudaSuccess) { release(); CUDA_CHECK(error); }
    }
}
```

При ошибке копирования выделенная память освобождается до исключения. Проверяется переполнение размера в байтах. Деструктор не генерирует исключения; ошибки рабочих вызовов CUDA обрабатываются `CUDA_CHECK`.

1.2 Представление MatrixView

MatrixView хранит невладеющий указатель и два размера. Индекс элемента (i, j) равен $i \cdot \text{ncols} + j$. Класс не выделяет и не освобождает память; его объекты передаются в кернел по значению. Тривиальная копируемость проверяется `static_assert`.

```
template<class AtomT> class MatrixView {
public:
    __host__ __device__ MatrixView() noexcept = default;
    __host__ __device__ MatrixView(AtomT* data, std::size_t rows, std::size_t cols)
noexcept
        : data_(data), nrows_(rows), ncols_(cols) {}
    __host__ __device__ std::size_t size() const noexcept { return nrows_*ncols_; }
    __host__ __device__ std::size_t nrows() const noexcept { return nrows_; }
    __host__ __device__ std::size_t ncols() const noexcept { return ncols_; }
    __host__ __device__ AtomT& operator[](std::size_t n) noexcept { return data_[n]; }
    __host__ __device__ const AtomT& operator[](std::size_t n) const noexcept { return
data_[n]; }
    __host__ __device__ AtomT& operator()(std::size_t i, std::size_t j) noexcept {
return data_[i*ncols_+j]; }
    __host__ __device__ const AtomT& operator()(std::size_t i, std::size_t j) const
noexcept { return data_[i*ncols_+j]; }
private:
    AtomT* data_ = nullptr;
    std::size_t nrows_ = 0, ncols_ = 0;
};
static_assert(std::is_trivially_copyable_v<MatrixView<float>>);
static_assert(std::is_trivially_copyable_v<MatrixView<const float>>);
```

Входные аргументы кернела имеют тип `MatrixView<const AtomT>`, поэтому запись в исходные матрицы через этот интерфейс запрещена компилятором. Методы индексации не проверяют границы; кернел проверяет номер строки и столбца до обращения к памяти.

1.3 Фасад Matrix и оператор умножения

Matrix объединяет `shared_ptr<Data<AtomT>>` и собственный `MatrixView`. Копии Matrix разделяют Data и GPU-память, но имеют разные объекты представления. Перемещение обнуляет представление источника. Обращение к `data()` перемещенного объекта вызывает исключение.

```
Matrix() : Matrix(0, 0) {}
Matrix(std::size_t rows, std::size_t cols)
    : data_(std::make_shared<Data<AtomT>>(checked_size(rows, cols))),
      view_(data_>data(), rows, cols) {}
Matrix(const Matrix&) = default;
Matrix& operator=(const Matrix&) = default;
Matrix(Matrix&& other) noexcept
    : data_(std::move(other.data_)), view_(std::exchange(other.view_, {})) {}
Matrix& operator=(Matrix&& other) noexcept {
    if (this != &other) {
        data_ = std::move(other.data_);
        view_ = std::exchange(other.view_, {});
    }
    return *this;
}

template<class AtomT>
Matrix<AtomT> operator*(const Matrix<AtomT>& a, const Matrix<AtomT>& b) {
    if (a.ncols() != b.nrows()) throw std::invalid_argument("A.cols must equal B.rows");
    Matrix<AtomT> result(a.nrows(), b.ncols());
    if (result.size()) {
        launch_matmul(a.read_view(), b.read_view(), result.view());
        CUDA_CHECK(cudaDeviceSynchronize());
    }
    return result;
}
```

Оператор проверяет согласованность размеров, создает результат и запускает вычисление. `cudaDeviceSynchronize` ожидает завершения и выявляет ошибки выполнения. Реализованы инстанцирования для `float` и `double`.

1.4 Двумерный CUDA-кernels

```
template<class AtomT>
__global__ void kernel_matmul_naive(MatrixView<const AtomT> a,
                                   MatrixView<const AtomT> b, MatrixView<AtomT> c) {
    const std::size_t row = static_cast<std::size_t>(blockIdx.y)*blockDim.y +
threadIdx.y;
    const std::size_t col = static_cast<std::size_t>(blockIdx.x)*blockDim.x +
threadIdx.x;
    if (row >= c.nrows() || col >= c.ncols()) return;
    AtomT sum = AtomT{0};
    for (std::size_t k = 0; k < a.ncols(); ++k)
        sum += a(row, k) * b(k, col);
    c(row, col) = sum;
}
```

Блок содержит 16×16 нитей. Размер сетки равен ($\text{ceil}(n / 16)$, $\text{ceil}(m / 16)$). Нити за границами результата завершаются без записи, поэтому поддерживаются размеры, не кратные 16, включая 127 и 129.

Каждая действующая нить последовательно выполняет k умножений и сложений. Всего производится порядка $2mnk$ арифметических операций; для квадратных матриц - порядка $2n^3$. Арифметическая сложность составляет $O(mnk)$, объем памяти для входов и выхода - $O(mk + kn + mn)$.

Наивный алгоритм не использует shared memory и не оптимизирует повторное чтение элементов плитками. Ожидаемое время зависит также от пропускной способности памяти, числа одновременно работающих нитей и накладных расходов запуска.

2 Модульное тестирование

Google Test формирует декартово произведение трех наборов размеров через Combine. Эталон рассчитывается как произведение Eigen::MatrixXf. Перед копированием на GPU данные явно преобразуются в row-major, поскольку MatrixXf по умолчанию использует column-major.

```
const Eigen::MatrixXf a_ref = ar, b_ref = br;
const Eigen::MatrixXf expected = a_ref*b_ref;
Matrix<float> a(m,k), b(k,n);
a.data().copy_from_host(ar.data()); b.data().copy_from_host(br.data());
const auto c = a*b;
ASSERT_EQ(c.nrows(), m); ASSERT_EQ(c.ncols(), n);
RowMatrix actual_row(m,n); c.data().copy_to_host(actual_row.data());
const Eigen::MatrixXf actual = actual_row;
EXPECT_TRUE(actual.isApprox(expected, 1e-5f));
EXPECT_LE((actual-expected).cwiseAbs().maxCoeff(), 1e-5f);
}
INSTANTIATE_TEST_SUITE_P(All1343Shapes, ProductTest,
    ::testing::Combine(::testing::Values(1,2,3,127,128,129,512),
        ::testing::Values(1,2,3,127,128,129,512),
        ::testing::Values(1,2,3,127,128,129,512)));
```

isApprox использует относительное сравнение норм, а не абсолютную погрешность. Поэтому дополнительно проверяется $\max|C - C_{ref}| \leq 10^{-5}$. Основной набор использует значения $q/16$, q от -8 до 8: произведения и суммы для $k \leq 512$ точно представимы в float. Отдельный тест проверяет не двоично-рациональные случайные входы малой амплитуды.

Дополнительные проверки: единичная и нулевая матрицы, несовместимые и пустые размеры, глубокое копирование Data, разделяемое копирование Matrix, перемещение, переполнение размеров, row-major индексация и double.

Фактический результат тестов пока отсутствует. Полнота подготовленного набора не означает, что CUDA-реализация уже проверена запуском.

3 Методика измерения производительности

Используется Google Benchmark. Для каждого размера выполняются три прогревочных умножения и семь повторов измерений. Для графиков выбирается медиана времени; для времени дополнительно показывается диапазон минимум-максимум. Сборка выполняется в Release, Eigen работает в одном CPU-потоке.

CUDA-бенчмарк действительно вызывает `operator*`. Его выделение памяти результата происходит до стартового CUDA Event, а освобождение - после конечного события. События создаются один раз до цикла. Входы копируются на GPU до измерений. Google Benchmark получает длительность между событиями через `SetIterationTime`.

```
KernelTimer timer;
for(auto _ : state) {
    ScopedKernelTimer measurement(timer);
    auto c=a*b; // Measure the actual operator; events inside exclude
allocation/free.
    benchmark::DoNotOptimize(c.data().data());
    state.SetIterationTime(timer.seconds());
}
```

Для Eigen результат выделяется заранее, затем выполняется `c.noalias() = a * b`. Время измеряется `steady_clock`. `EIGEN_RUNTIME_NO_MALLOC` запрещает динамические выделения внутри измеряемой операции, включая временные буферы. Буферы упаковки размещаются на стеке; лимит стека Windows увеличен до 64 МиБ. Проверка запрета работает и в Release.

Полученное ускорение относится только к вычислениям. Оно не описывает полное время приложения с передачей данных между CPU и GPU. Малые размеры чувствительны к стоимости запуска и таймера; медиана не устраняет все систематические погрешности.

4 Результаты вычислительного эксперимента

Эксперимент на GPU не выполнен. Планируемая видеокарта по информации исполнителя - RTX 2060. Ее фактические параметры, версия драйвера, CUDA, процессор и времена будут записаны при запуске; сейчас они не представлены как измеренные.

После успешного запуска `scripts/run.ps1` сохраняет `tests.xml`, `benchmarks.json` и сведения об окружении. `scripts/plot.py` читает только реальные записи Google Benchmark, отклоняет ошибки, пропущенные размеры и неполные серии. На их основе создаются таблица, графики времени и ускорения.

Отчет с экспериментальной частью создается командой `scripts/make_report.py --results <папка запуска> --output <report.pdf>`. Генерация окончательного отчета запрещена при неуспешных тестах или отсутствии измерений.

4.1 План анализа

Для каждого соседнего размера вычисляется показатель $p = \ln(t_2/t_1) / \ln(n_2/n_1)$. Для кубического роста при удвоении n время увеличивается примерно в 8 раз, $p \approx 3$. На малых размерах накладные расходы и неполная загрузка GPU могут дать другое поведение.

Ускорение определяется как $S(n) = t_{\text{Eigen}}(n) / t_{\text{CUDA}}(n)$. При $S > 1$ вычислительная часть CUDA быстрее, при $S < 1$ быстрее Eigen. До получения данных нельзя утверждать наличие ускорения или его максимальное значение.

ЗАКЛЮЧЕНИЕ

Подготовлена реализация умножения матриц с разделяемым владением памятью и невладеющим представлением. Двумерная сетка CUDA распределяет элементы результата между нитями. Разделяемая память в кернеле не используется.

Для завершения работы необходимо собрать проект и выполнить тесты и измерения на компьютере с NVIDIA GPU. До этого корректность CUDA-сборки и производительность не заявляются подтвержденными. Настоящий проект отчета не является завершенным экспериментальным отчетом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Тарасов С. А. Практическая работа №2. Материалы задания (work2.pdf).
2. NVIDIA. CUDA C++ Programming Guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
3. Eigen. DenseBase::isApprox. https://libeigen.gitlab.io/eigen/docs-3.4/classEigen_1_1DenseBase.html
4. Google Test. Advanced Guide. <https://google.github.io/googletest/advanced.html>
5. Google Benchmark. User Guide. https://google.github.io/benchmark/user_guide.html
6. Eigen. FAQ: runtime allocation checks. <https://libeigen.gitlab.io/pages/faq/>